

SPRINT 3 BACKLOG DOCUMENT

Project: TripRoute (Travel Assistant)

Team 5 Members:

- Sameer Maayiz
- Dongwan Kim
- Mainuddin

Overview

Building on Sprint 2's foundation, Sprint 3 focused on migrating from ChatGPT to Gemini AI for faster and more reliable recommendations, implementing a production-ready FastAPI server with session management, developing a complete frontend interface with interactive map visualization and route display, and enhancing the route optimization engine to support all four transportation modes (driving, walking, bicycling, transit). The result is a fully functional end-to-end TripRoute system that provides AI-powered trip planning with real-time Google Maps grounding, structured recommendation extraction, and optimized multi-modal route visualization.

List of User Stories

- As a user, I want faster AI responses so that I can plan my trips more efficiently without long wait times.
- As a user, I want to see my trip recommendations displayed on an interactive map with optimized routes for different transportation modes.
- As a developer, I want to migrate from ChatGPT to Gemini AI to leverage Google Search grounding for up-to-date place information.
- As a developer, I want a production API server with session management so that multiple users can chat simultaneously.
- As a user, I want a complete web interface that combines chat, map visualization, and route details in one cohesive experience.
- As a developer, I want route optimization across all four transportation modes (driving, walking, bicycling, transit).
- As a developer, I want comprehensive documentation for API endpoints, installation, and troubleshooting to support deployment and maintenance.

Task Breakdown

User Story 1 (Gemini AI Migration)

- Replace OpenAI ChatGPT API with Google Gemini 2.5 Flash Lite
- Implement Google Search grounding for real-time place information
- Configure Gemini streaming for faster response times (1-2 seconds vs 10-30 seconds)
- Update system prompts to enforce Google Maps grounding on every place recommendation

User Story 2 (Production API Server)

- Build FastAPI server (server.py) with CORS middleware
- Implement in-memory session storage for Phase 1 deployment
- Create /chat endpoint with streaming responses
- Create /optimize endpoint for route generation
- Implement /session endpoint for session initialization
- Add background task for structured data extraction

User Story 3 (Frontend Interface Development)

- Design responsive 3-column layout (map, places list, chat)
- Implement chat interface with message history and streaming support
- Integrate Google Maps JavaScript API for interactive map display
- Build places list view with collapsible route details
- Add transport mode switching (driving, walking, bicycling, transit)
- Implement draggable column splitters for customizable layout
- Create print functionality for trip itineraries

User Story 4 (Route Optimization Enhancement)

- Enhance optimization.py to support all 4 transportation modes
- Implement Place ID normalization for reliable place matching
- Add special transit handling (driving optimization first, then leg-by-leg transit)
- Implement place enrichment with Google Maps Place Details API
- Generate detailed step-by-step directions for each route
- Output optimized routes to 4 separate JSON files per transport mode

User Story 5 (Structured Data Extraction)

- Implement Pydantic models for TripRouteRecommendations schema
- Configure Gemini structured output with JSON schema
- Extract trip metadata (city, start/end dates, theme, transport preference)

- Parse place recommendations with all required fields
- Link extracted Place IDs from grounding metadata to recommendations

User Story 6 (Documentation)

- Create comprehensive SETUP_GUIDE.md with environment configuration
- Write RUNNING_LOCALLY.md for development workflow
- Document API endpoints and response formats
- Create TROUBLESHOOTING.md for common issues
- Write INSTALL_AND_RUN.md for quick deployment
- Document frontend features in LAYOUT_FEATURES.md

Assigned Team Members

- **Sameer Maayiz:** Frontend UI (demo.html), Chat Interface, Map Integration, Responsive Design
- **Dongwan Kim:** Backend API (server.py), Gemini Integration, Session Management, Environment Configuration
- **Mainuddin:** Route Optimization (optimization.py), Google Maps APIs, Multi-modal Transit Support, DevOps & Documentation

Estimated Time & Priority

- Gemini AI Migration: (High priority, 1.5 weeks)
- Production API Server: (High priority, 2 weeks)
- Frontend Interface: (High priority, 2.5 weeks)
- Route Optimization Enhancement: (High priority, 1.5 weeks)
- Structured Data Extraction: (Medium priority, 1 week)
- Documentation: (Medium priority, 1 week)
- Testing & Debugging: (High priority, 1 week)

Progress Status

- Gemini AI Migration – Completed
- Google Search Grounding Integration – Completed

- Production API Server (server.py) – Completed
- Session Management – Completed
- /chat Endpoint with Streaming – Completed
- /optimize Endpoint – Completed
- Structured Data Extraction – Completed
- Frontend Interface (demo.html) – Completed
- Map Integration – Completed
- Chat UI – Completed
- Places List View – Completed
- Transport Mode Switching – Completed
- Route Optimization for 4 Modes – Completed
- Place ID Normalization – Completed
- Transit Special Handling – Completed
- Comprehensive Documentation – Completed
- Testing & Debugging – Completed

Summary

Sprint 3 successfully delivered a production-ready TripRoute system with complete AI-powered trip planning capabilities. The migration from ChatGPT to Gemini AI reduced response times from 10-30 seconds to 1-2 seconds while providing more accurate, up-to-date place information through Google Search grounding. The new FastAPI server enables multi-user support with session management, and the comprehensive frontend interface provides an intuitive user experience combining chat, map visualization, and detailed route information across four transportation modes.